

Setor de Educação Profissional e Tecnológica

Tecnologia em Análise e Desenvolvimento de Sistemas

DS141 - Estruturas de Dados II

12 - Grafos Direcionados e Menor Caminho

Prof. Alex Kutzke

Agosto de 2021

Tradução adaptada dos slides presentes em [1]. Imagens e exemplos também retirados do mesmo material.

[1] <<https://algs4.cs.princeton.edu/lectures/keynote/34HashTables.pdf>>

Roteiro

Introdução

Implementação

Busca em Profundidade e em Largura

Menor Caminho

Introdução

Grafos Direcionados

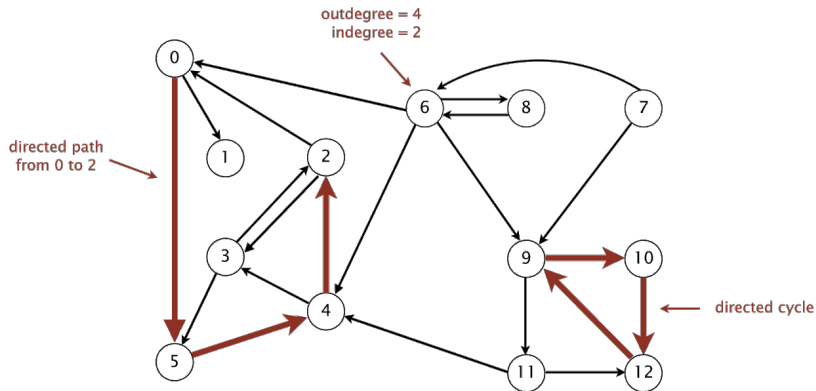
Como vimos na última aula, grafos **não direcionados** são aqueles que, entre outras propriedades, tem sua relação de “conectado a” como simétrica. Ou seja:

Se v é conectado à w , então w é conectado à v .

Grafos direcionados (ou Digrafos), por sua vez, não possuem essa relação simétrica.

Grafos Direcionados

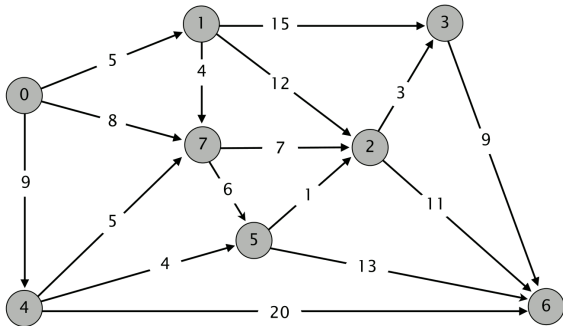
Dessa forma, as arestas em Grafos Direcionados devem indicar uma **direção**.



Fonte: (SEdgeWICK; WAYNE, 2014)

Arestas valoradas

Um outra variação da modelagem de grafos é quando as arestas possuem **valores** (ou pesos).

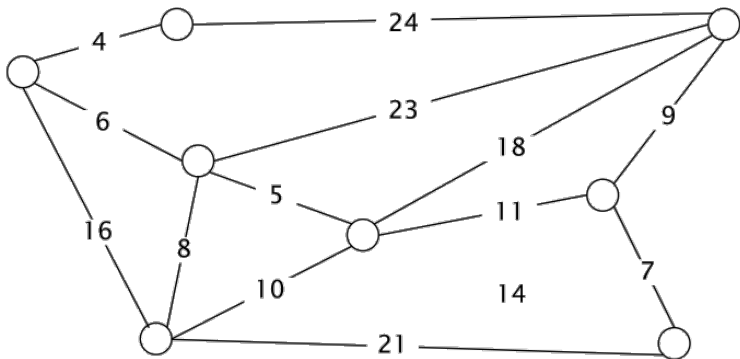


Fonte: (SEdgeWICK; WAYNE, 2014)

Pesos nas arestas permitem a modelagem de um conjunto maior de problemas.

Arestas valoradas

É importante notar que tanto grafos **não direcionados** quanto **direcionados** podem ter arestas com pesos.



Fonte: (SEDGEWICK; WAYNE, 2014)

Implementação

Estrutura para Listas de Adjacências de Grafos Direcionados com arestas valoradas

```
1  struct node {
2      int to;                // destination node of edge
3      link next;            // next element in adjacency list
4      double wt;            // cost of edge
5  };
6  typedef struct node *link;
7
8  struct digraph {
9      int V;                // number of vertices
10     int E;                // number of edges
11     link *adj;            // adjacency list
12 };
13
14 typedef struct graph *Graph;
15 typedef struct { int from; int to; } Edge;
```

Criação de um Grafo

```
1 Digraph DIGRAPHinit(int V) {
2     int v;
3     Digraph G = malloc(sizeof *G);
4     G->V = V; G->E = 0;
5     G->adj = malloc(V * sizeof(*G->adj));
6     if (G->adj == NULL){
7         fprintf(stderr, "Out of memory in GRAPHinit()\n");
8         exit(EXIT_FAILURE);
9     }
10
11     for (v = 0; v < V; v++) G->adj[v] = NULL;
12
13     return G;
14 }
```

Inserção de Aresta

```
1 void DIGRAPHinsertE(Digraph G, Edge e, double d) {  
2     int v = e.from;  
3     int w = e.to;  
4     G->adj[v] = NEW(w, G->adj[v], d);  
5     G->E++;  
6 }
```

Busca em Profundidade e em Largura

Funcionamento em Grafos Direcionados

Os algoritmos de Busca em Profundidade e em Largura utilizados em Grafos Não Direcionados também podem ser utilizados em Grafos Direcionados.

Detalhe Busca em Largura

Um detalhe importante é que, em grafos com arestas valoradas, a busca em largura pode **não** entregar o menor caminho correto (considera apenas o número de arestas e não o peso).

Para o cálculo de menor caminho em grafos direcionados e com arestas valoradas, temos outros algoritmos.

Menor Caminho

Algoritmo de Dijkstra

Para grafos direcionados com arestas valoradas (sem pesos negativos) podemos utilizar o Algoritmo de Dijkstra.



Edsger W. Dijkstra
Turing award 1972

Fonte: (SEDGEWICK; WAYNE, 2014)

O Algoritmo de Dijkstra encontra o menor caminho entre um vértice inicial e todos os outros alcançáveis por ele.

Relaxação de Aresta

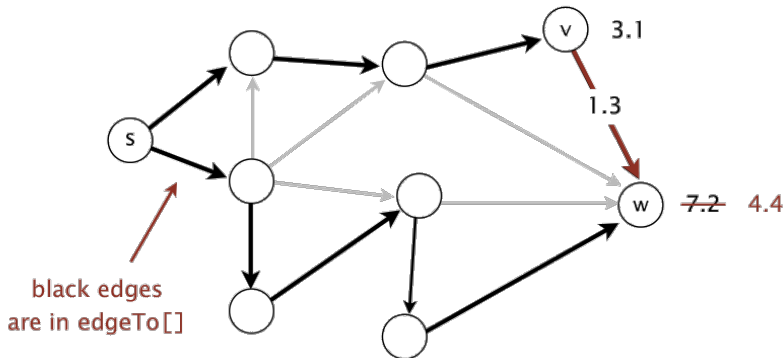
O Algoritmo de Dijkstra se baseia no conceito de **relaxação de aresta**:

Relaxação da aresta $e = v \rightarrow w$

- $\text{distTo}[v]$: é o comprimento do menor caminho **conhecido** de s até v ;
- $\text{distTo}[w]$: é o comprimento do menor caminho **conhecido** de s até w ;
- $\text{edgeTo}[w]$: é a última aresta no menor caminho **conhecido** de s até w ;
- Se $e = v \rightarrow w$ entrega um menor caminho para w através de v então atualiza $\text{distTo}[w]$ e $\text{edgeTo}[w]$.

Relaxação de Aresta

$v \rightarrow w$ successfully relaxes



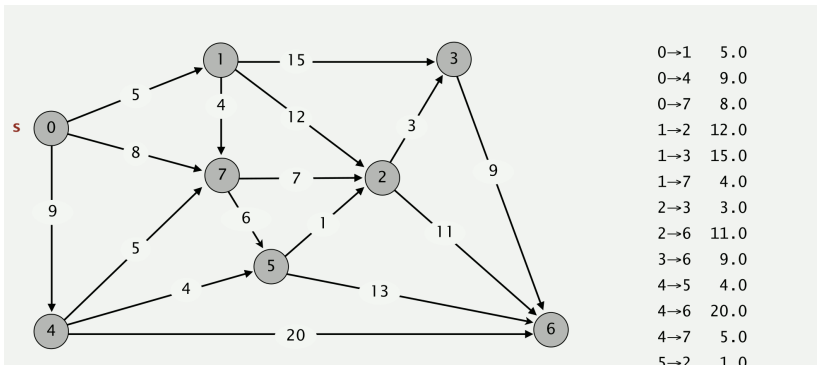
Fonte: (SEDGWICK; WAYNE, 2014)

Algoritmo de Dijkstra

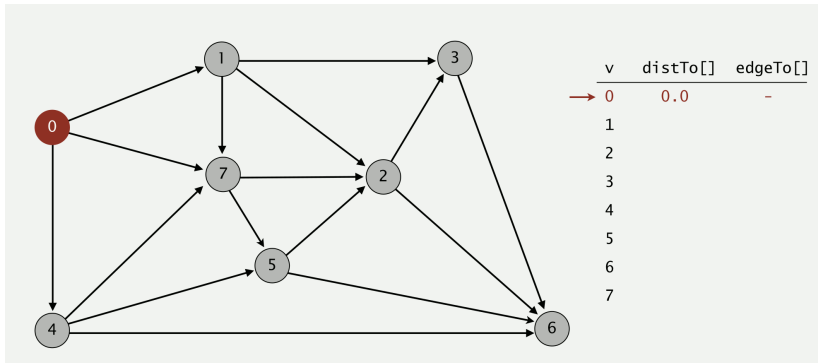
O Algoritmo tem os seguintes passos:

1. Escolher o vértice v não visitado com menor distância a partir de s até o momento;
2. Marcar vértice como visitado;
3. Realizar a relaxação de todas as arestas que apontam para o vértice v .

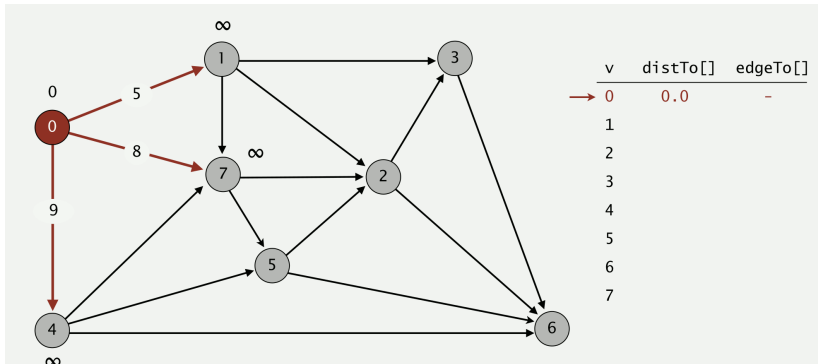
Exemplo Algoritmo de Dijkstra



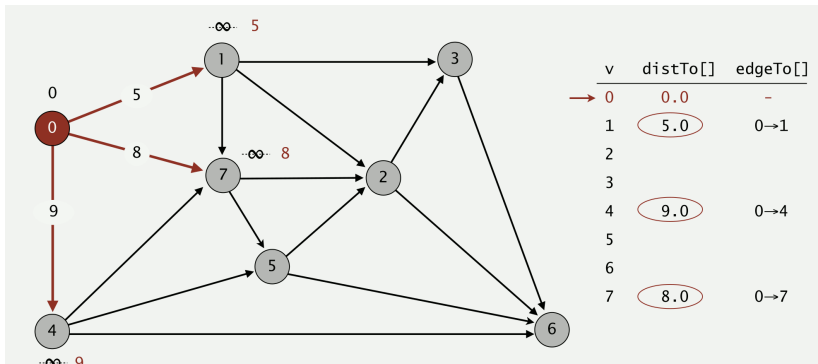
Exemplo Algoritmo de Dijkstra



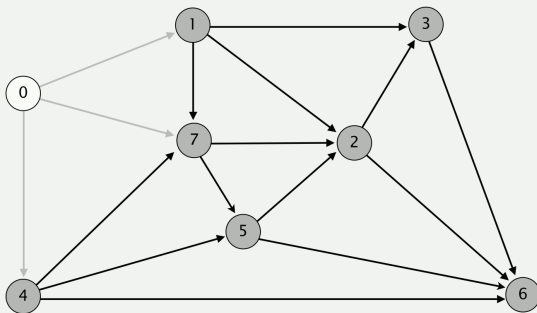
Exemplo Algoritmo de Dijkstra



Exemplo Algoritmo de Dijkstra

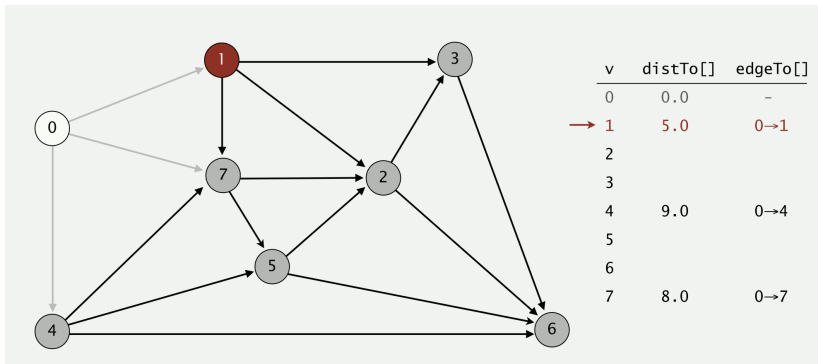


Exemplo Algoritmo de Dijkstra

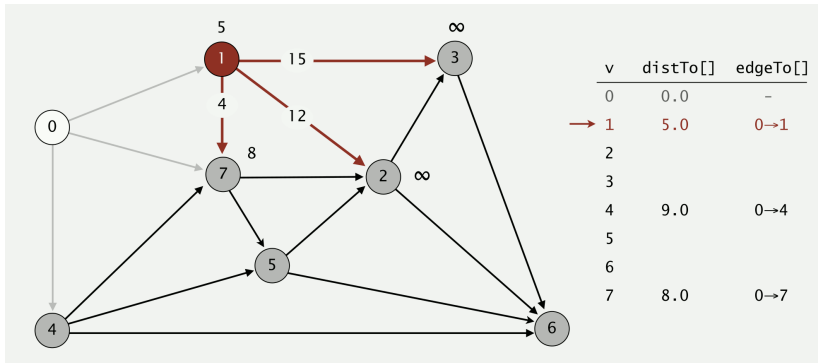


v	distTo[]	edgeTo[]
0	0.0	-
1	5.0	0→1
2		
3		
4	9.0	0→4
5		
6		
7	8.0	0→7

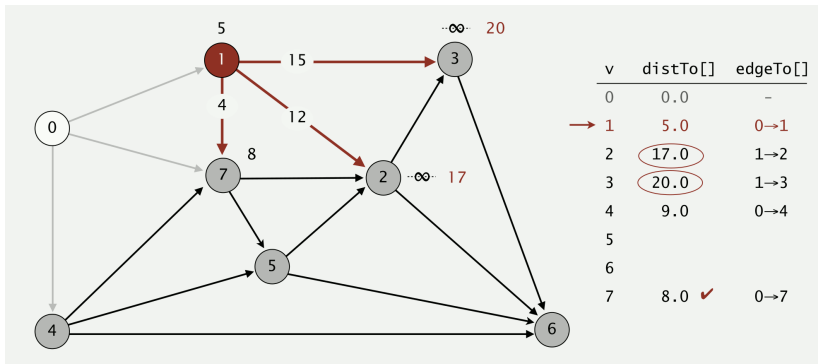
Exemplo Algoritmo de Dijkstra



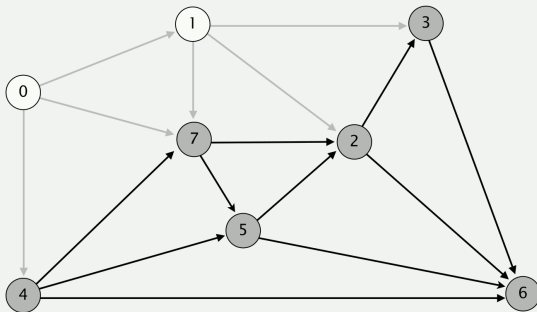
Exemplo Algoritmo de Dijkstra



Exemplo Algoritmo de Dijkstra

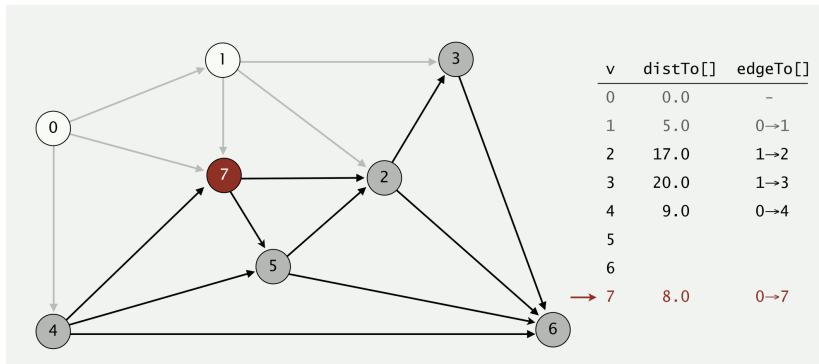


Exemplo Algoritmo de Dijkstra

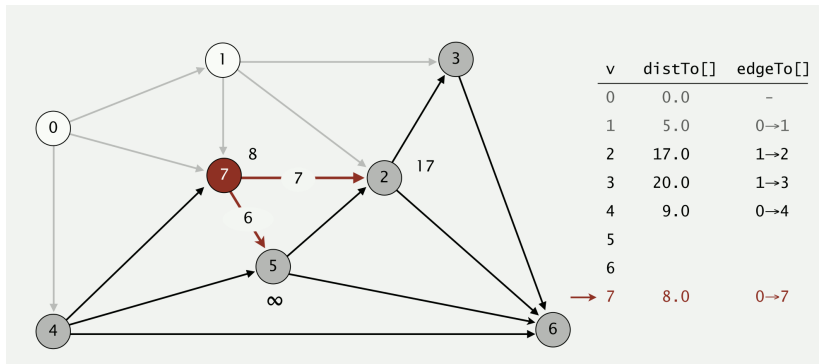


v	distTo[]	edgeTo[]
0	0.0	-
1	5.0	0→1
2	17.0	1→2
3	20.0	1→3
4	9.0	0→4
5		
6		
7	8.0	0→7

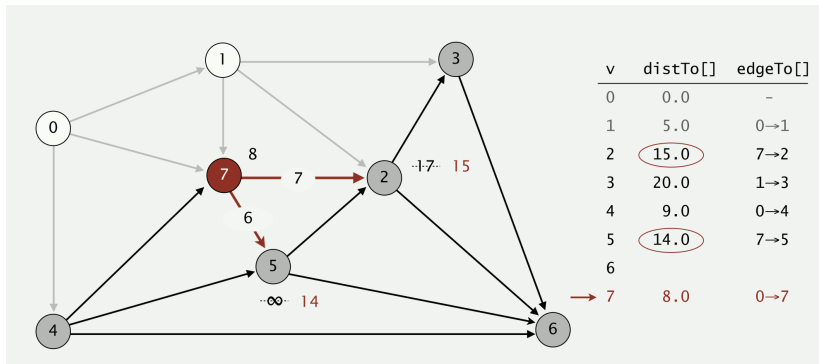
Exemplo Algoritmo de Dijkstra



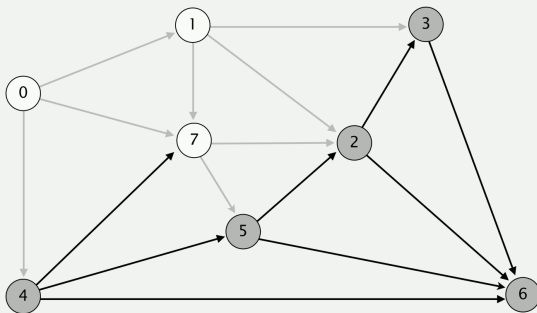
Exemplo Algoritmo de Dijkstra



Exemplo Algoritmo de Dijkstra

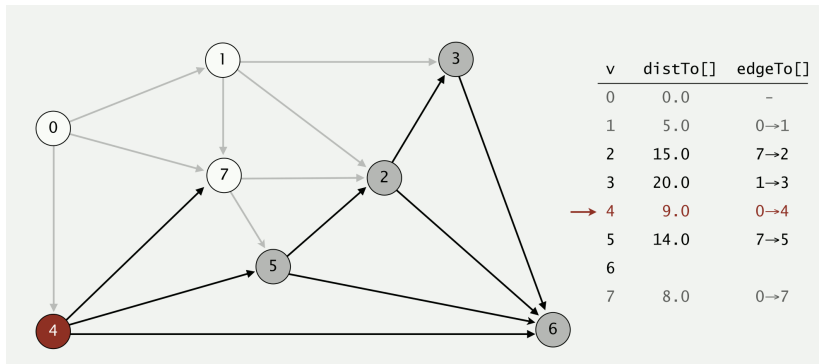


Exemplo Algoritmo de Dijkstra

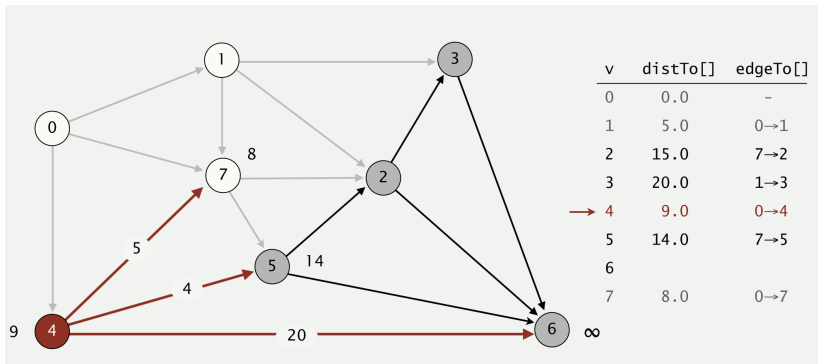


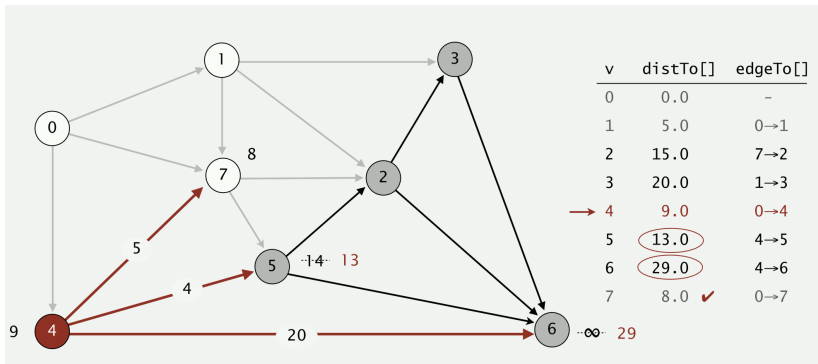
v	distTo[]	edgeTo[]
0	0.0	-
1	5.0	0→1
2	15.0	7→2
3	20.0	1→3
4	9.0	0→4
5	14.0	7→5
6		
7	8.0	0→7

Exemplo Algoritmo de Dijkstra

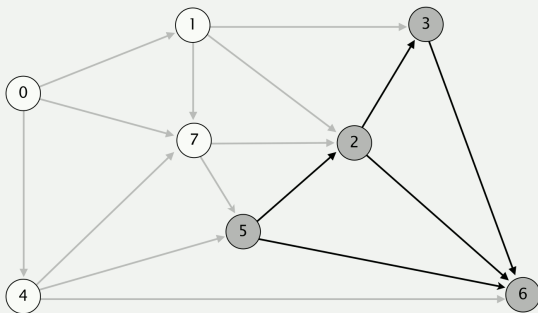


Exemplo Algoritmo de Dijkstra



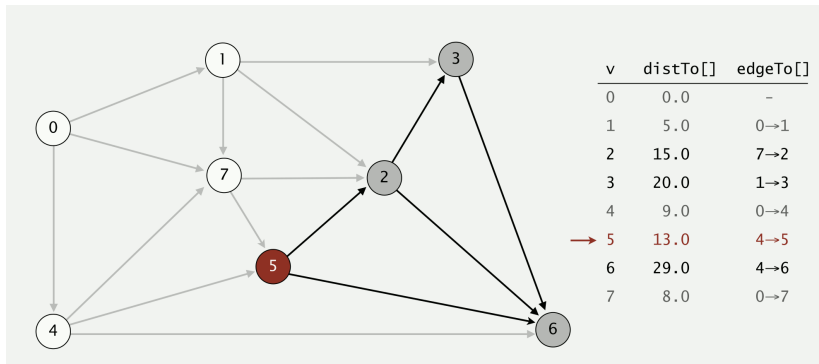


Exemplo Algoritmo de Dijkstra

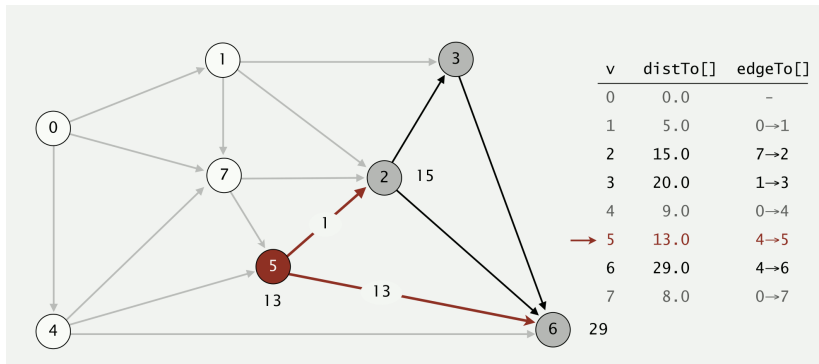


v	distTo[]	edgeTo[]
0	0.0	-
1	5.0	0→1
2	15.0	7→2
3	20.0	1→3
4	9.0	0→4
5	13.0	4→5
6	29.0	4→6
7	8.0	0→7

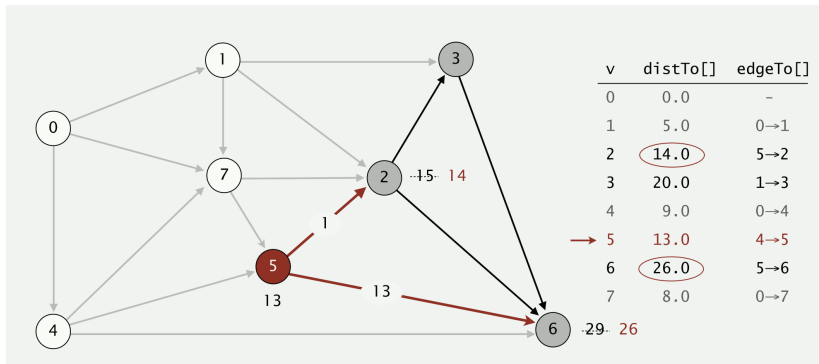
Exemplo Algoritmo de Dijkstra



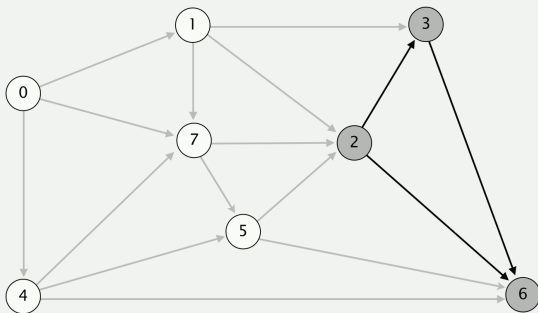
Exemplo Algoritmo de Dijkstra



Exemplo Algoritmo de Dijkstra

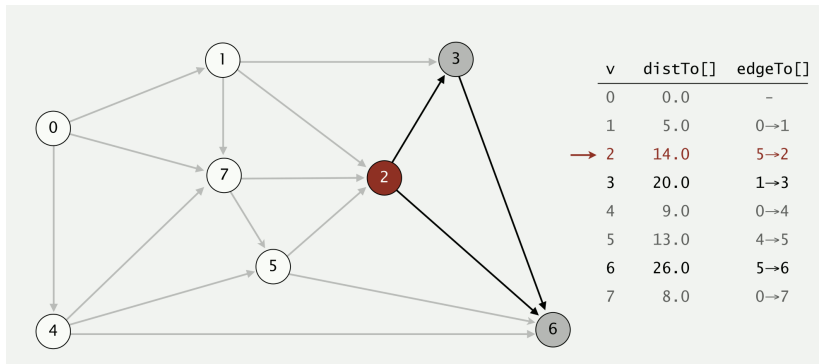


Exemplo Algoritmo de Dijkstra

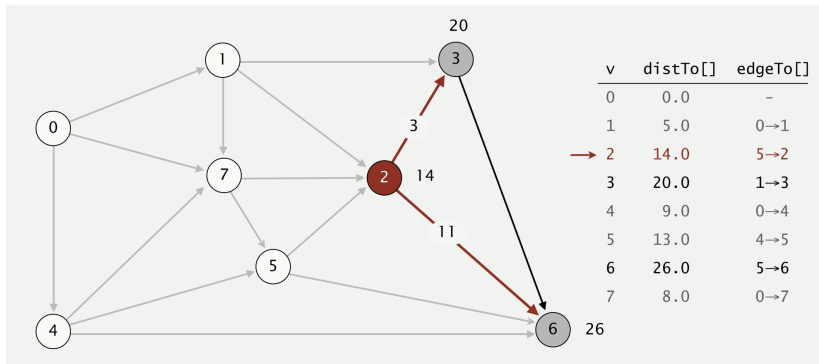


v	distTo[]	edgeTo[]
0	0.0	-
1	5.0	0→1
2	14.0	5→2
3	20.0	1→3
4	9.0	0→4
5	13.0	4→5
6	26.0	5→6
7	8.0	0→7

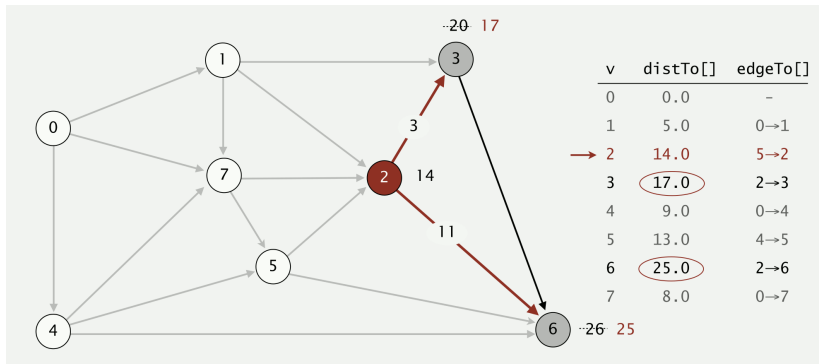
Exemplo Algoritmo de Dijkstra



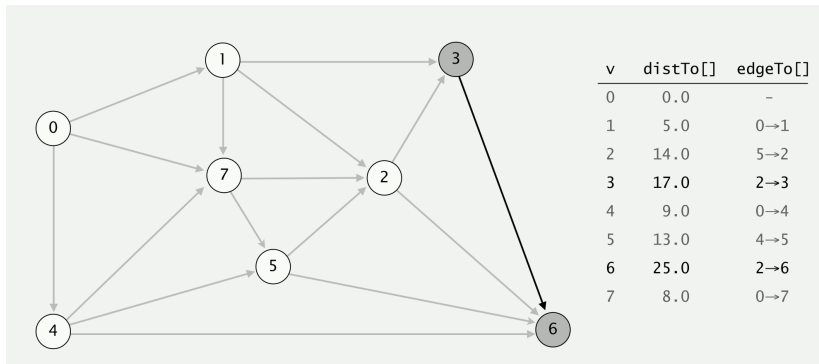
Exemplo Algoritmo de Dijkstra



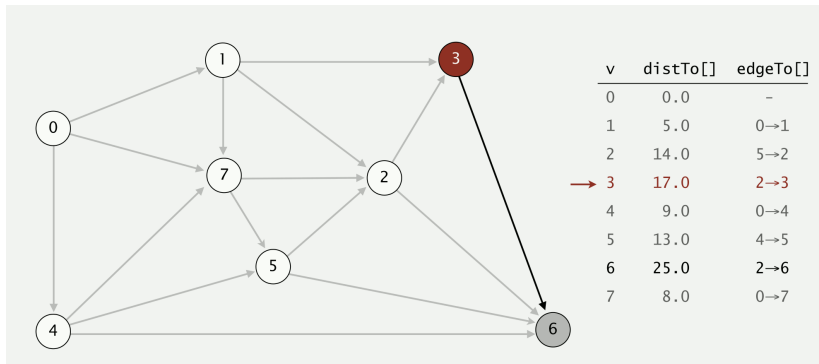
Exemplo Algoritmo de Dijkstra



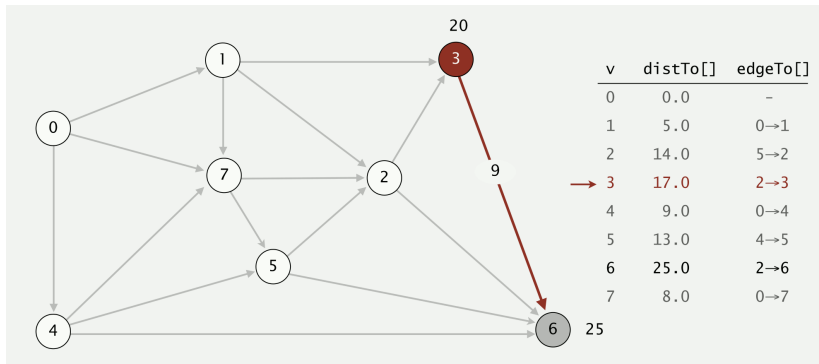
Exemplo Algoritmo de Dijkstra



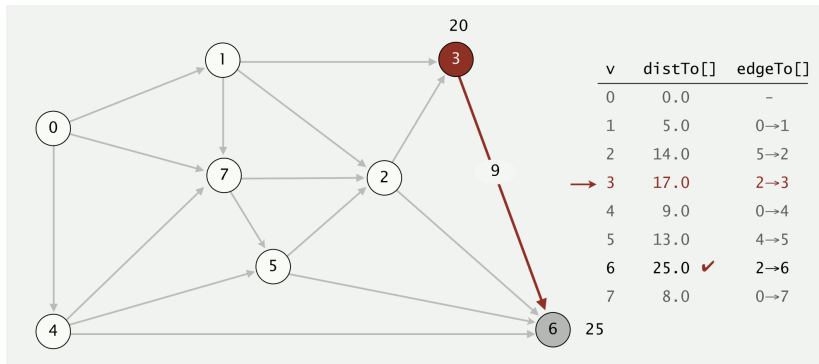
Exemplo Algoritmo de Dijkstra



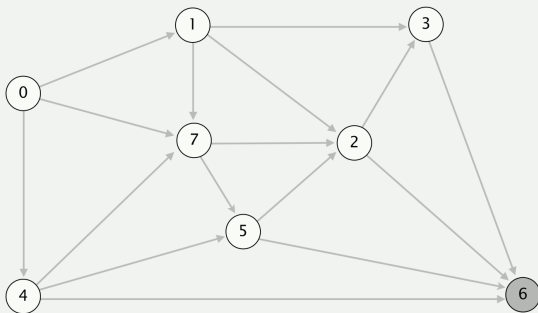
Exemplo Algoritmo de Dijkstra



Exemplo Algoritmo de Dijkstra

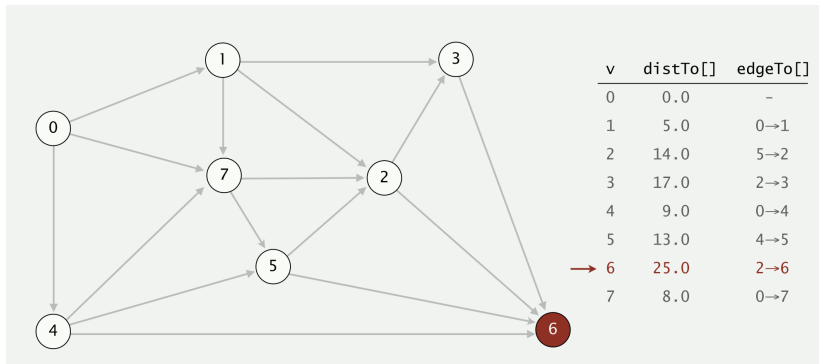


Exemplo Algoritmo de Dijkstra

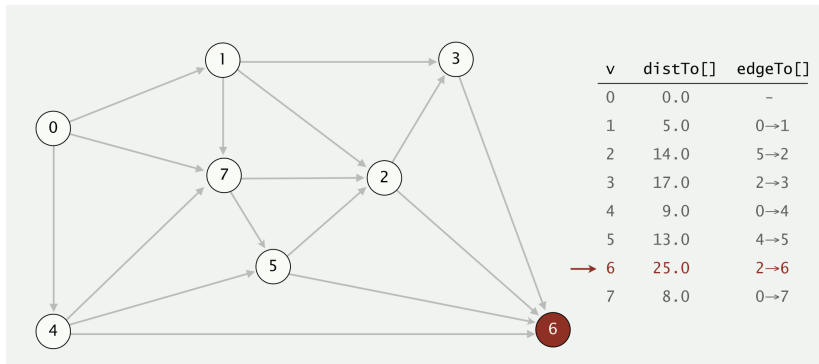


v	distTo[]	edgeTo[]
0	0.0	-
1	5.0	0→1
2	14.0	5→2
3	17.0	2→3
4	9.0	0→4
5	13.0	4→5
6	25.0	2→6
7	8.0	0→7

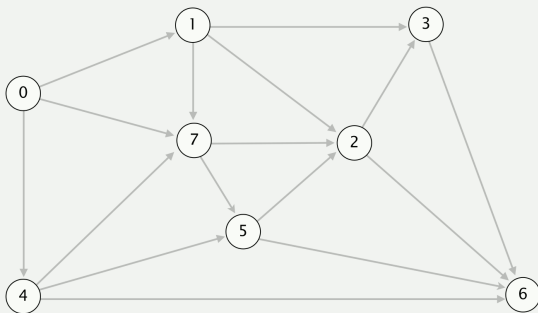
Exemplo Algoritmo de Dijkstra



Exemplo Algoritmo de Dijkstra

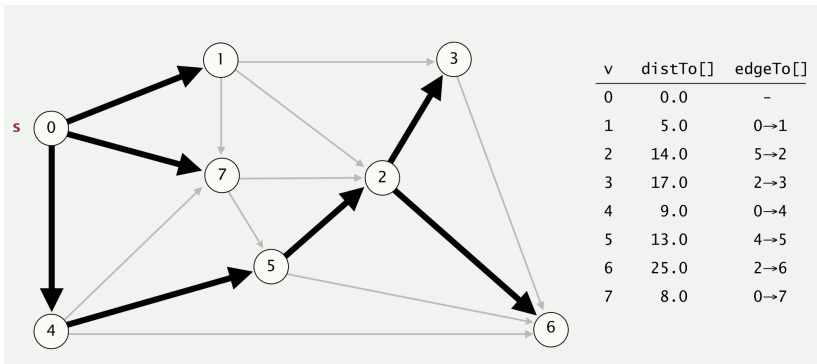


Exemplo Algoritmo de Dijkstra



v	distTo[]	edgeTo[]
0	0.0	-
1	5.0	0→1
2	14.0	5→2
3	17.0	2→3
4	9.0	0→4
5	13.0	4→5
6	25.0	2→6
7	8.0	0→7

Exemplo Algoritmo de Dijkstra



Referências utilizadas na apresentação

 SEDGEWICK, R.; WAYNE, K. *Algorithms: Part I*. [S.l.]: Addison-Wesley Professional, 2014.